Optymalizacja Kodu na różne architektury: Zadanie 1

Jakub Gucwa

22 kwietnia 2026

1 Cel zadania

Celem tego zadania jest zaimplementowanie funkcji przetwarzającej duże ciągi tekstowe. Funkcja powinna wykonywać następujące operacje:

- Usuwanie znaków spoza zakresu drukowalnych.
- Zamiana sekwencji białych znaków na pojedynczą spację.
- Konwersja wszystkich liter na małe litery.
- Zamiana interpunkcji na przecinki.
- Eliminacja duplikatów wyrazów występujących jeden po drugim.

2 Wersja 1 - implementacja bazowa

Wersja `normalize1.cpp` jest rozwiązaniem bazowym napisanym z użyciem klasy `std::string`. Algorytm przechodzi liniowo po całym wejściu i buduje nowy napis `result`. Dodatkowo utrzymywane są dwa bufory słów: `current_word` (aktualnie czytane słowo) oraz `last_word` (ostatnie zatwierdzone słowo), co pozwala usuwać duplikaty sąsiadujących wyrazów.

Kolejne etapy działania są następujące:

- Dla każdego znaku sprawdzane jest, czy jest białym znakiem (`isspace`). Jeżeli tak, to sekwencja wielu białych znaków zostaje sprowadzona do pojedynczej spacji.
- Znaki spoza zakresu drukowalnego ASCII (mniejsze od 32 lub większe od 126) są pomijane. Ważne jest to, aby to był krok drugi, ponieważ niektóre białe znaki są spoza tego zakresu, a powinny być zamienione na spację.
- Znaki interpunkcyjne (`ispunct`) są zamieniane na przecinek.
- Każdy zachowany znak jest konwertowany do małej litery (`tolower`) i dopisywany do wyniku.

- Po zakończeniu słowa porównywane są `current_word` i `last_word`. Jeśli są identyczne, ostatnio dopisane słowo jest cofane z wyniku.

Na końcu wykonywana jest jeszcze kontrola dla ostatniego słowa (gdy wejście nie kończy się spacją). Złożoność czasowa jest liniowa względem długości tekstu, ale wersja tworzy wiele pomocniczych obiektów i często realokuje pamięć.

Listing 1: Kod funkcji `normalize_string` w wersji 1

```
std::string normalize_string(std::string input){
    std::string result = "";
    std::string current_word = "";
    std::string last_word = "";
    unsigned int n = input.size(), i = 0;
    unsigned char ascii_val;
    while(i<n){
        ascii_val = (unsigned char)input[i];
        if(isspace(ascii_val)){
            if(!result.empty() && result.back() != '␣'){
                if(!current_word.empty() && current_word ==
                    last_word){
                    result.resize(result.size() -
                        current_word.size());
                } else {
                    result += '␣';
                }
                last_word = current_word;
                current_word = "";
            }
            i++; continue;
        }

        if(ascii_val < 32 || ascii_val >126){
            i++; continue;
        }
        if(ispunct(ascii_val)){
            ascii_val = ',';
        }
        current_word += tolower(ascii_val);
        result += tolower(ascii_val);
        i++;

    }
    if(!current_word.empty() && current_word == last_word){
            result.resize(result.size() - current_word.size
                ());
        }

    return result;
}
```

3 Wersja 2 - optymalizacja zarządzania pamięcią

Wersja `normalize2.cpp` zachowuje tę samą logikę normalizacji co wersja 1, ale optymalizuje zarządzanie pamięcią wyjścia. Najważniejsze zmiany:

- Bufor wyniku jest od razu rezerwowany przez `result.resize(input.size())`, dzięki czemu zapis odbywa się przez indeks i ogranicza koszt wielokrotnego dopisywania.
- Wprowadzono licznik `result_size`, który określa faktyczną długość danych w buforze i pozwala wykonywać operacje cofania bez kosztownych modyfikacji całego napisu.
- Dodano licznik `curr_word_size`, co upraszcza usuwanie zduplikowanego słowa przez jednorazowe zmniejszenie długości wyniku.

W porównaniu z wersją 1 zmniejsza to narzut alokacji i kopiowania, przy zachowaniu identycznego efektu końcowego.

Listing 2: Fragment optymalizacji funkcji `normalize_string` w wersji 2

```
std::string result;
result.resize(input.size());
int result_size = 0;
// ...
result[result_size++] = tolower(ascii_val);
result_size -= curr_word_size;
// ...
// na koniec:
result.resize(result_size);
```

4 Wersja 3 - normalizacja in-place

Wersja `normalize3.cpp` idzie dalej i wykonuje normalizację **in-place**, czyli bez tworzenia osobnego napisu wynikowego. Kluczowe zmiany względem wersji 2:

- Funkcja przyjmuje referencję `std::string &input` i modyfikuje ten sam bufor.
- Zastosowano dwa indeksy: `i` do odczytu oraz `j` do zapisu, co pozwala nadpisywać dane wejściowe wynikiem.
- Po zakończeniu przetwarzania wykonywane jest `input.resize(j)`, które odcina nieużywaną końcówkę.

Ta wersja redukuje zużycie pamięci i liczbę kopiowań, co zwykle poprawia wydajność dla dużych plików.

Listing 3: Kluczowe zmiany w funkcji `normalize_string` w wersji 3

```
void normalize_string(std::string &input){
    unsigned int i = 0, j = 0;
    // ...
    if(j>0 && input[j-1] != '␣'){
        if(!current_word.empty() && current_word ==
            last_word){
            j -= curr_word_size;
        } else {
            input[j++] = '␣';
        }
    // ...
    input[j] = tolower(ascii_val);
    current_word += input[j];
    curr_word_size++;
    i++; j++;
    // ...
    input.resize(j);
}
```

5 Wersja 4 - implementacja biblioteczna

Wersja `normalize4.cpp` zmienia podejście implementacji - opiera się na algorytmach STL.

- Wstępna transformacja wszystkich znaków realizowana jest przez `std::transform` (białe znaki na spację, interpunkcja na przecinek, litery na małe).
- Usuwanie znaków spoza zakresu drukowalnego wykonuje `std::remove_if`.
- Redukcja wielu spacji do jednej realizowana jest przez `std::unique` z własnym predykatem.
- Usuwanie kolejnych duplikatów słów oparto o `std::string_view`, `std::find` i `std::copy`.

W tej wersji znacznie bardziej widoczne są kolejne kroki, ale kosztem wiekszej liczby przebiegów po danych wejściowych, co sprawia, że dla bardzo dużych plików wersja ta jest wolniejsza od pozostałych.

Listing 4: Funkcja `normalize_string` w wersji 4

```

void normalize_string(std::string &input){
    std::transform(input.begin(), input.end(),input.begin(),
        [](unsigned char c) -> char {
            if (isspace(c))  return '␣';
            if (ispunct(c))  return ',';
            return (char)tolower(c);
        });

    input.resize(
        std::remove_if(input.begin(), input.end(),
            [](unsigned char c){ return c < 32 || c > 126;
                })
        - input.begin());

    input.resize(
        std::unique(input.begin(), input.end(),
            [](char a, char b){ return a == '␣' && b == '␣';
                })
        - input.begin());

    auto word_begin = input.begin();
    auto write_end  = input.begin();
    std::string_view last_word;

    while (word_begin != input.end()) {
        auto word_end = std::find(word_begin, input.end(), '
            ␣');
        std::string_view current(&*word_begin, (size_t)(
            word_end - word_begin));
```

```

        if (current != last_word) {
            if (write_end != input.begin()) *write_end++ = ' ';
            auto new_word_start = write_end;
            write_end = std::copy(word_begin, word_end, write_end);
            last_word = std::string_view(&*new_word_start, current.size());
        }

        word_begin = word_end;
        if (word_begin != input.end()) ++word_begin;
    }
    input.erase(write_end, input.end());
}
```

6 Wersja 5 - optymalizacja z wykorzystaniem podstawowych tablic znaków

Wersja `normalize5.cpp` przechodzi z C++-owych napisów na bufor `char*` i funkcje z biblioteki C. Algorytm jest oparty o ten z wersji 3, ale wprowadzono następujące zmiany:

- Operacje wykonywane są na surowej tablicy znaków, co daje większą kontrolę nad pamięcią i ogranicza narzut abstrakcji `std::string`.
- Porównywanie słów do detekcji duplikatów wykorzystuje `memcmp`, a kopiowanie `memcpy`.
- Ostatnie słowo zapamiętywane jest w buforze `last_word_buf`, co pozwala szybko sprawdzić, czy bieżące słowo powtarza poprzednie.

Przez podejście niskopoziomowe algorytm jest szybszy, ale kod jest mniej czytelny.

Listing 5: Kluczowy fragment funkcji `normalize_string` w wersji 5

```c
size_t normalize_string(char* p, size_t n){

    // ...
    char* last_word_buf = (char*)malloc(n + 1);

    while (i < n) {
        c = (unsigned char)p[i];

        if (isspace(c)) {
            if (j > 0 && p[j-1] != '␣') {
                if (curr_word_size > 0 && curr_word_size == last_word_len &&
                    memcmp(p + j - curr_word_size, last_word_buf, curr_word_size) == 0) {
                    j -= curr_word_size;
                } else {
                    memcpy(last_word_buf, p + j - curr_word_size, curr_word_size);
                    last_word_len = curr_word_size;
                    p[j++] = '␣';
                }
                curr_word_size = 0;
            }
            i++;
            continue;
        }

        if (c < 32 || c > 126) { i++; continue; }
        if (ispunct(c)) c = ',';

        p[j++] = (char)tolower(c);
        curr_word_size++;
        i++;
    }
    // ...
    free(last_word_buf);
    p[j] = '\0';
    return j;
}
```

7 Wersja 6 - przetwarzanie wielowątkowe

Wersja `normalize6.cpp` rozszerza wersję 5 o przetwarzanie wielowątkowe. Najważniejsze zmiany:

- Wejście jest dzielone na fragmenty, a każdy fragment przetwarza osobny wątek (funkcja `partial_normalize`).
- Granice fragmentów są przesuwane do granic słów (po białych znakach), aby nie zaczynać fragmentu w środku wyrazu.
- Po zakończeniu pracy wątków fragmenty są scalane do jednego bufora, z dodatkową obsługą duplikatów słów na granicach fragmentów.

W porównaniu z wersją 5 celem jest skrócenie czasu działania na dużych danych przez wykorzystanie wielu rdzeni procesora. Kosztem jest większa złożoność kodu i konieczność poprawnego scalenia wyników cząstkowych.

Listing 6: Podział na oddzielne zadania dla wątków w `normalize_string` w wersji 6

```
for (size_t i = 1; i < num_threads; i++) {
    size_t pos = i * part_size;
    while (pos < n && !isspace((unsigned char)p[pos])) pos
        ++;
    while (pos < n &&  isspace((unsigned char)p[pos])) pos
        ++;
    frag_starts[i] = pos;  // granica na slowie
}
for (size_t i = 0; i < num_threads; i++) {
    size_t flen = frag_end - frag_starts[i];
    threads[i] = std::thread([=]() {
        frag_sizes[i] = partial_normalize(
            p + frag_starts[i], flen);
    });
}
for (size_t i = 0; i < num_threads; i++) threads[i].join();
```

8 Analiza profilera (gprof)

Każdą implementację skompilowano z flagą `-pg` i uruchomiono 400-krotnie na pliku `regulartests/test1M`. Wyniki zebrano narzędziem `gprof`.

Zestawienie wyników

Wersja	Dominująca funkcja	% czasu	Łączny czas	ms/call
1	`normalize_string` (std::string)	91.9%	1.11 s	2.55
2	`normalize_string` (std::string)	91.3%	1.27 s	2.90
3	`normalize_string` (std::string&)	81.2%	1.17 s	2.38
4	`normalize_string` (std::string&)	93.2%	1.17 s	2.73
5	`normalize_string` (char*)	81.5%	0.81 s	1.65
6	`partial_normalize` (4 wątki)	80.9%	0.94 s	0.475/wątek

Wnioski z profilowania

- We wszystkich wersjach praktycznie cały czas procesu spędza się wewnątrz funkcji normalizującej (80–93%), co potwierdza, że główne wąskie gardło leży w logice przetwarzania znaków, nie w I/O ani inicjalizacji.
- **Wersja 1 vs 2**: Wersja 2 wykazała nieco wyższy czas z profilerem (1.27 s vs 1.11 s) – narzut instrumentacji `-pg` zaburza pomiar przy dużej liczbie małych operacji na `std::string`.
- **Wersja 3**: Operacja in-place (bez tworzenia osobnego bufora) obniżyła ms/call do 2.38 – mniej alokacji pamięci przekłada się na realną poprawę.
- **Wersja 4**: Podejście STL (kilka przebiegów `transform`/`remove_if`/`unique`) okazało się wolniejsze od wersji 3 (2.73 ms/call) – kolejne przebiegi po danych generują dodatkowy narzut cache.
- **Wersja 5**: Przejście na surowy bufor `char*` z `memcmp`/`memcpy` przyniosło największą poprawę spośród implementacji jednowątkowych (1.65 ms/call, łączny czas 0.81 s).
- **Wersja 6**: Wielowątkowość (4 wątki) skróciła realny czas wykonania do 0.355 s (pomiar bez `-pg`). Gprof mierzy 80.9% czasu w `partial_normalize` wywoływanym 1600 razy (400 iter. $\times$ 4 wątki). Łączny czas gprof (0.94 s) jest sumą czasu wszystkich wątków, nie rzeczywistego czasu wykonania, z czego pozorne spowolnienie względem wersji 5 w tabeli powyżej.

9 Analiza profilera (perf)

Dla każdej wersji wykonano pomiar `perf` (`record -e cycles:u -F 999`) na danych `regulartests/test1M`. Poniżej zestawiono czas pojedynczego uruchomienia programu oraz główny hotspot z raportu `perf report`.

Wersja	Czas [s]	Top % (perf)	Najgorętszy symbol
1	1.562987	73.18%	`normalize_string(std::string)`
2	1.586685	70.17%	`normalize_string(std::string)`
3	1.442242	70.12%	`normalize_string(std::string&)`
4	1.196079	82.10%	`normalize_string(std::string&)`
5	0.966707	70.45%	`normalize_string(char*, size_t)`
6	0.311479	88.89%	`normalize_string/partial_normalize`

Wnioski z perf

- We wszystkich implementacjach główny koszt CPU pozostaje w funkcji normalizującej; udział funkcji wynosi około 70–82% (dla wersji 1–5).
- Dla wersji 1–5 drugim wyraźnym kosztem jest `__ctype_tolower_loc@plt`, co jest niczym niespodziewanym, ponieważ każdy zapisywany znak wywołuje `tolower`.
- Wersja 6 ma najniższy czas ściany (0.311 s).

10 Wykresy

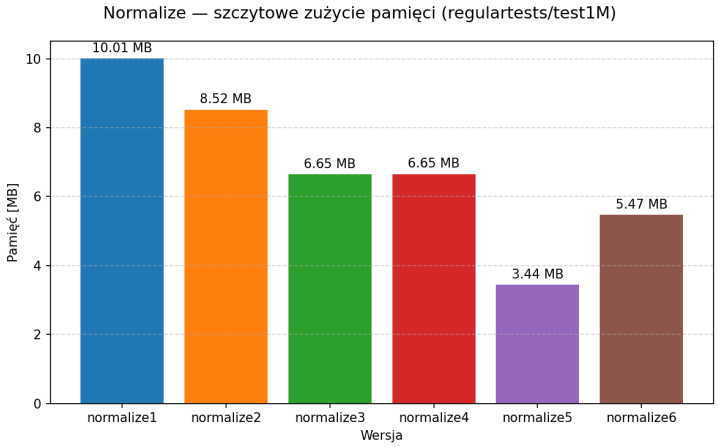

Figure 1: Wykres użycia pamięci.

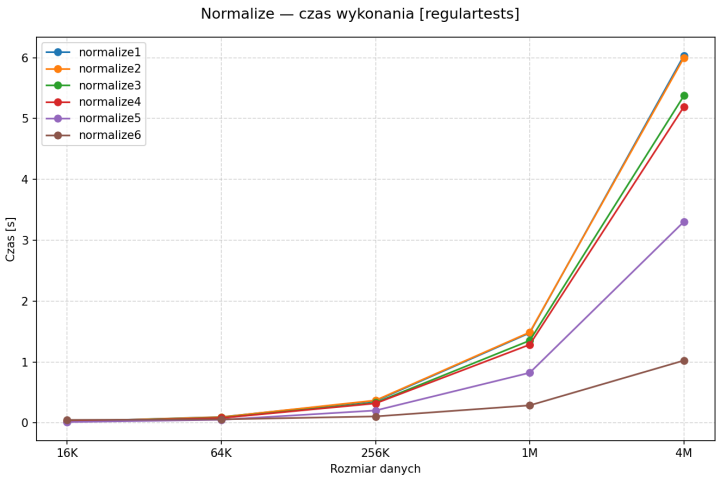

Figure 2: Czas działania poszczególnych implementacji – dane regularne.

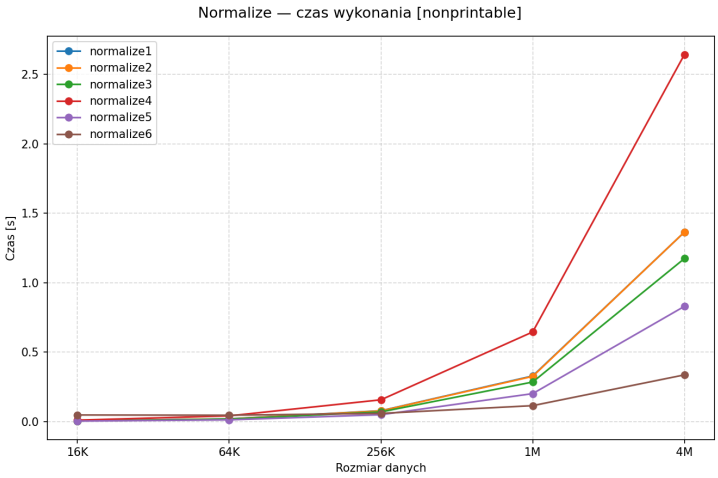

Figure 3: Czas działania poszczególnych implementacji – głównie niedrukowalne znaki.

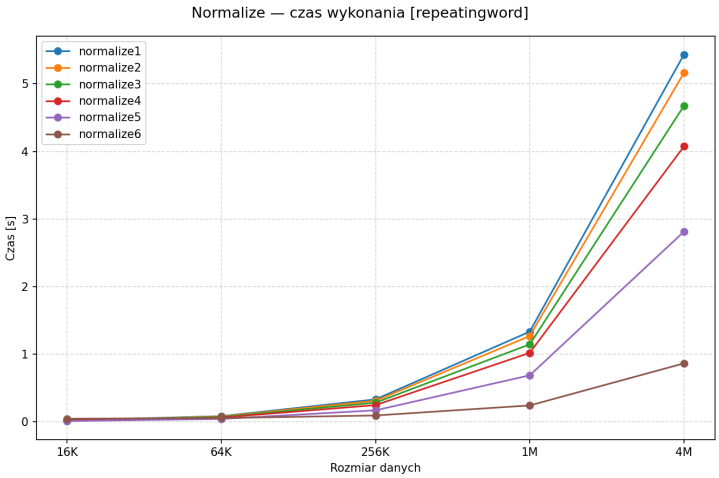

Figure 4: Czas działania poszczególnych implementacji – głównie powtarzające się wyrazy.

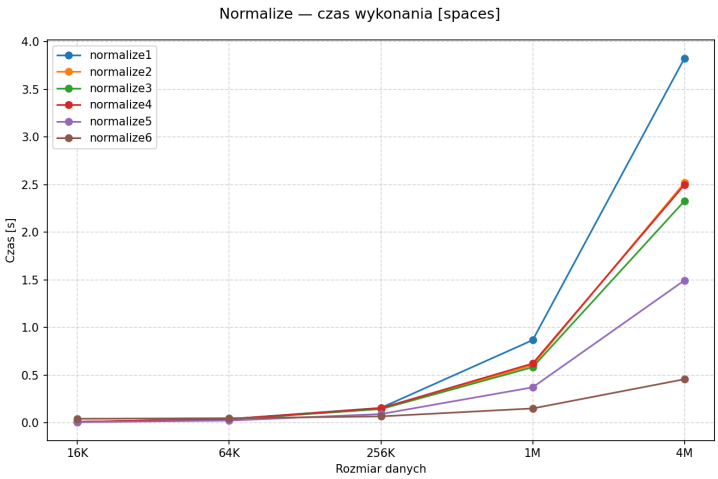

Figure 5: Czas działania poszczególnych implementacji – głównie białe znaki.